QUALIDEV_VERSIONING — COMPTE RENDU

Récap Git — Tous les TPs

TD 1 → TP 6 · init, collaboration, reset, rebase -i, bisect, hooks, sous-modules

Format ◎ scénario → ▶ séquence → ★ à retenir

Polsinelli Johan · pj509414

◆ Récap Git — Tous les TPs (QUALIDEV_VERSIONING)

Fiches de révision par TP et par exercice, au format : ◎ **scénario** → ▶ **séquence de commandes** → ★ **à retenir**.

Sommaire

Fichier	TP	Thème	Compétence clé
TD1-init-collaboration.md	TD 1	Init local, SSH, GitLab, workflow binôme	init / remote / push / MR / rebase / reflog
TP1-notions-avancees.md	TP 1	Notions avancées	annuler un rebase / tag perdu après amend
TP2-git-reset.md	TP 2	`git reset`	ORIG_HEAD / HEAD~n / déplacer des branches
TP3-reecrire-historique.md	TP 3	Réécrire l'historique	`rebase -i` : squash, reword, drop, edit
TP4-debugger-avec-git.md	TP 4	Déboguer avec Git	`bisect` / `bisect run` / `blame`
TP5-hooks.md	TP 5	Les hooks Git	`pre-commit` / `commit-msg` / hook `update` serveur
TP6-sous-modules.md	TP 6	Sous-modules / multi-dépôts	`submodule add` / pointeurs / clone récursif

---

◆ Table globale des commandes par catégorie

Observer

Commande	Rôle
`git status`	État du dossier : modifs, staging, conflits
`git log --oneline --graph --all --decorate`	Historique dessiné, compact, toutes branches, avec tags
`git log -1 --format="%H"`	Récupérer le sha complet d'un commit
`git show <ref>`	Détail d'un commit ou d'un tag
`git reflog`	Boîte noire : TOUTES les actions, même resets/rebases

Commande	Rôle
`git blame <fichier>`	Qui a écrit chaque ligne, et dans quel commit
`git tag`	Lister les tags

Naviguer

Commande	Rôle
`git switch <branche>`	Changer de branche
`git switch -c <branche>`	Créer ET basculer sur une branche
`git checkout <sha/tag>`	Aller sur un commit précis → **HEAD détachée**

Enregistrer

Commande	Rôle
`git add <fichier>`	Mettre dans le staging (aussi : marquer un conflit résolu)
`git commit -m "..."`	Créer un commit
`git commit --amend`	⚠ Ne « modifie » pas : crée un **nouveau** commit (nouveau sha)

Annuler / réparer

Commande	Rôle
`git reset --hard <sha>`	Branche + fichiers remis à l'état du commit. ⚠ Perd les modifs
`git reset --soft / --mixed`	Bouge le pointeur sans toucher aux fichiers (mixed = défaut)
`git reset --hard ORIG_HEAD`	Annuler le dernier merge/rebase/reset
`git reset --hard HEAD~n`	Reculer de n commits
`git merge --abort`	Annuler un merge en cours
`git rebase --abort` / `--continue`	Abandonner / poursuivre un rebase (après résolution de conflit)

Branches & fusion

Commande	Rôle
`git merge <branche>`	Fait entrer `<branche>` dans la branche **courante**
`git merge --no-ff`	Force un commit de merge même si fast-forward possible
`git rebase <branche>`	Replante la branche courante au bout de `<branche>` (réécrit les sha !)

Commande	Rôle
`git rebase -i`	Rebase interactif : squash/reword/drop/edit/réordonner
`git branch -d <branche>`	Supprimer une branche fusionnée (sécurisé)
`git branch -D <branche>`	Supprimer de force (⚠ commits potentiellement orphelins)
`git branch -f <branche> <cible>`	Déplacer une branche sans s'y placer

Distant (GitLab)

Commande	Rôle
`git clone <url>`	Copier un dépôt distant en local
`git remote add/rename`	Ajouter / renommer un remote
`git push -u origin <branche>`	Pousser + lier la branche locale au distant
`git push origin --all`	Pousser toutes les branches
`git pull`	Récupérer + fusionner les modifs distantes

Tags

Commande	Rôle
`git tag <nom> [sha]`	Créer un tag **léger** (simple pointeur)
`git tag -a <nom> -m "..."`	Créer un tag **annoté** (objet complet : auteur, date, message) — celui des releases
`git tag -d <nom>`	Supprimer un tag
`git tag -f <nom> [HEAD]`	**Déplacer** un tag existant

Bisect / hooks / sous-modules

Commande	Rôle
`git bisect start / good / bad / reset`	Recherche dichotomique du commit fautif
`git bisect run <script>`	Bisection automatisée (script exit 0/1)
`.git/hooks/pre-commit`, `commit-msg`	Hooks locaux (non versionnés, bloquants si exit ≠ 0)
`hooks/update`	Hook **serveur** : contrôle chaque ref poussée
`git submodule add <url> <chemin>`	Ajouter un sous-module
`git clone --recurse-submodules`	Cloner projet + sous-modules
`git submodule update --init`	Récupérer les sous-modules après un clone normal

★ Les 5 lois d'or apprises dans ces TPs

1. **Le sha = empreinte du contenu + message + parent.** Toute « modification » (amend, rebase...) crée un nouveau commit. L'ancien devient orphelin.
2. **Un commit orphelin n'est pas perdu** tant qu'il est référencé (tag, branche) ou visible dans le **reflog**.
3. **Un tag ne bouge jamais tout seul** : historique réécrit → tag à déplacer à la main (`-f`).
4. **Sens des opérations** : `merge` / `rebase` agissent sur la branche **courante**. Se tromper de sens → `git reset --hard ORIG_HEAD` et on recommence.
5. **Le reflog voit tout.** C'est la boîte noire anti-catastrophe.

TD 1 — Initialisation locale, GitLab et workflow collaboratif

Exercice 1 — Initialisation locale et README

◎ **Scénario :** créer un dépôt local depuis zéro, s'identifier, faire son premier commit.

▶ Séquence :

```
mkdir pj509414 && cd pj509414      # 1. créer le dossier de travail
git init -b main                   # 2. initialiser Git, branche principale = main
git config --local user.name "Johan Polsinelli"    # 3. identité (ce dépôt seulement)
git config --local user.email "johan.polsinelli@etu.unice.fr"
git add README.md                  # 4. mettre le README dans le staging
git commit -m "Initial commit: Création du README"  # 5. premier commit
```

★ À retenir :

- `git init -b main` crée le dépôt avec `main` directement (évite le renommage `master` → `main`).
- `--local` = config valable **uniquement dans ce dépôt** (`--global` = tous tes dépôts).
- Le cycle de base : **modifier** → **`add`** → **`commit`**.

Exercice 2 — Première synchronisation GitLab (SSH + push)

◎ **Scénario :** prouver son identité au serveur via SSH, lier le dépôt local au dépôt distant, pousser.

▶ Séquence :

```
ssh-keygen -t ed25519 -C "johan.polsinelli@etu.unice.fr"  # 1. générer la paire de clés SSH
eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519       # 2. charger la clé dans l'agent
cat ~/.ssh/id_ed25519.pub   # 3. copier la clé PUBLIQUE → la coller dans GitLab (SSH Keys)
ssh -T git@baowlab.sountrust.fr   # 4. tester : "Welcome to GitLab, @pj509414!" ✔

git remote add origin git@baowlab.sountrust.fr:pj509414/td1.git  # 5. lier le distant
git push --set-upstream origin main   # 6. pousser + lier main local ↔ main distant
```

★ À retenir :

- Clé **privée** = reste chez toi (`id_ed25519`) ; clé **publique** = donnée au serveur (`.pub`).
- `origin` = surnom du dépôt distant. `-u` / `--set-upstream` = mémorise la liaison, ensuite `git push` suffit.

Exercice 3 — Collaboration avancée et workflow (binôme, MR, rebase, reflog)

◆ Préparation et config (Q1–Q5)

```
git config --global init.defaultBranch main   # Q1: main par défaut pour tout nouveau dépôt
git branch -m master main                     # Q1: renommer la branche courante si besoin

echo "[Ticket] #" > ~/.gitmessage.txt         # Q2: modèle de message de commit
git config --global commit.template ~/.gitmessage.txt

git remote rename origin old-origin           # Q4: renommer l'ancien serveur (BaowLab)
git remote add origin git@iut-git.unice.fr:pj509414/pj509414.git  # Q4: nouveau = GitLab IUT
git push -u origin --all                      # Q5: pousser TOUTES les branches d'un coup
```

◆ Q3 — Dépôt vide vs avec README :

- **Vide** : aucun commit côté serveur → il attend ton push, tout roule.
- **Avec README** : GitLab crée un commit initial → les deux historiques divergent → push refusé (`non-fast-forward`). Il faut alors `git pull --rebase` ou `--allow-unrelated-histories`.

◆ Rôles GitLab : binôme + enseignant = **Developer** (peut pousser) ; groupe TP = **Reporter** (lecture seule).

◆ Travail chez le binôme (Q6–Q12)

```
cd .. && git clone git@iut-git.unice.fr:ce504439/ce504439.git binome_ce504439  # Q6
cd binome_ce504439
git switch -c feature/amelioration-readme     # Q7: branche de travail isolée
# Q8: deux modifs → deux commits "A" et "B"
git add README.md && git commit -m "A"
git add README.md && git commit -m "B"
git push -u origin feature/amelioration-readme  # Q9: pousser LA BRANCHE (pas main)
```

→ Q10/Q11 : création puis traitement de la **Merge Request** sur GitLab (revue → **Merge**).

◆ Pourquoi pousser sur une branche et pas directement `main` ? Isoler le travail en cours, ne pas casser la branche stable, permettre la **revue de code** et la CI/CD avant intégration.

◆ Le grand cycle develop → suppression → tag → rebase (Q13–Q27)

```
cd ../pj509414
git switch -c develop                         # Q13: branche develop en local
git add README.md && git commit -m "A'"       # Q13: 2 commits A' et B'
git add README.md && git commit -m "B'"

git switch main && git pull origin main       # Q14: récupérer le travail du binôme (fast-forward)
git log --all --decorate --oneline --graph    # Q15: voir l'arbre des 2 branches

git log -1 --format="%H" develop              # Q16: ⚠ SAUVEGARDER le sha de B' (ex: ae3325b...)
git switch main
```

```
git branch -D develop                          # Q17: suppression FORCÉE → commits orphelins

git tag v01 ae3325b349a64ad1a1c38cadc1b5a8b38ff324a4   # Q19: le tag ré-ancre les commits !
git checkout v01                               # Q21: → HEAD DÉTACHÉE (pas sur une branche)
git switch -c develop                          # Q22: recréer develop depuis le tag
git tag -d v01                                 # Q23: le tag n'est plus utile

git rebase main                                # Q24: replanter develop au bout de main
  # conflits → éditer, virer les marqueurs, puis pour CHAQUE conflit :
  git add README.md && git rebase --continue

git switch main && git merge develop           # Q27: intégrer (fast-forward après rebase)
git push origin main
```

★ À retenir

1. **Q18 — Orphelin ≠ perdu.** Après `branch -D`, les commits A'/B' disparaissent du graphe car plus rien ne les référence. Mais ils existent encore dans la base d'objets.
2. **Q20 — Un tag remplace une branche comme point d'ancrage.** Le graphe fait réapparaître les commits dès que `v01` pointe dessus.
3. **Q21 — HEAD détachée** = être sur un commit sans branche. Normal pour visiter ; pour travailler, créer une branche (`switch -c`).
4. **Q25 — Le rebase réécrit les sha** : A'/B' rejoués au-dessus de `main` = nouveaux sha. Historique linéaire.
5. **Q28 — Le reflog retrouve n'importe quel commit « perdu »** même sans avoir noté le sha : chercher la ligne `commit: B'` et son sha `ae3325b`.
6. **Conflit pendant un rebase** ≠ conflit pendant un merge : on valide avec `git rebase --continue` (pas `git commit`).

TP 1 — Notions Git avancées

Exercice 1 — Bug après rebase (exo_2_1)

◎ **Scénario :** après `git rebase master` sur develop, un bug apparaît (2 liens écrasés sur une ligne). Était-il là avant, ou est-ce le rebase ? Il faut **restaurer l'avant-rebase**, vérifier, puis merger develop dans master proprement et supprimer develop.

▶ Séquence :

```
git reflog                          # 1. retrouver l'état d'avant : sha juste avant "rebase (start)"
git reset --hard 20135bb            # 2. restaurer develop AVANT le rebase
   # variante juste après le rebase : git reset --hard ORIG_HEAD

# → TESTER LE SITE ICI : bug parti = le rebase l'a introduit ; bug présent = il préexistait

git switch master                   # 3. aller sur la branche RECEVEUSE
git merge develop                   # 4. merger (conflit dans hello.html)
#    → éditer : garder les 2 contenus, virer <<<<<<< ======= >>>>>>>, liens sur 2 lignes
git add hello.html                  # 5. marquer le conflit résolu
git commit                          # 6. valider la fusion (message "Merge branch 'develop'")
git branch -d develop               # 7. supprimer develop
git log --graph --oneline           # 8. vérifier : master seul, historique propre
```

★ À retenir :

- **Le rebase ne fusionne pas, il recopie** : les commits de develop sont rejoués avec de nouveaux sha — une mauvaise résolution automatique peut y introduire le bug.
- `ORIG_HEAD` = position avant la dernière opération risquée (merge/rebase/reset). Juste après un rebase → pointe sur l'avant-rebase.
- **Sens du merge** : pour mettre develop dans master, on est sur master. `git merge master` depuis develop dit « Déjà à jour » après un rebase (master ⊂ develop) — ce n'est PAS un bug, juste le mauvais sens.
- Porte de secours : `git merge --abort` pour annuler un merge en cours.

Exercice 2 — Tag perdu après amend (SITE_V03)

◎ **Scénario :** le collègue tague le commit « médaille d'or » (`SITE_V03`), puis fait `git commit --amend` pour corriger le message. Résultat : `git log` ne montre plus le tag, mais `git tag` le liste toujours. Pourquoi, et comment réparer ?

▶ Séquence :

```
git log --oneline --decorate   # 1. constater : tag absent du log
git tag                        # 2. constater : tag pourtant listé
git show SITE_V03 --oneline    # 3. diagnostic : le tag pointe sur l'ANCIEN sha
git reflog                     # 4. preuve : ligne "commit (amend)"
```

```
git tag -f -a SITE_V03 -m "Version 03 du site" HEAD   # 5. déplacer/recréer le tag annoté sur HEAD
   # ou en 2 étapes : git tag -d SITE_V03  puis  git tag -a SITE_V03 -m "..."

git log --oneline --decorate   # 6. vérifier : (HEAD -> master, tag: SITE_V03) ✔
```

★ À retenir :

1. **Amend ≠ modification.** Le sha = empreinte du contenu **+ du message** → changer le message crée un nouveau commit. L'ancien sort de l'historique → `git log` ne le voit plus, ni le tag accroché dessus.
2. **Un tag ne suit jamais tout seul** : il reste scotché à son sha d'origine. Historique réécrit → tag à déplacer soi-même avec `-f`.
3. **Tag annoté (`-a -m`)** = vrai objet Git (auteur, date, message) → c'est le type à utiliser pour les releases. Tag léger (`git tag v1`) = simple étiquette.
4. ⚠ Si le collègue avait déjà **pushé** le tag : il faudrait aussi `git push --force` en prévenant l'équipe (réécriture d'historique partagé).

TP 2 — git reset

> **Fil rouge du TP :** `git reset --hard ORIG_HEAD` est le « Ctrl+Z » des opérations qui réécrivent ou déplacent l'historique (merge, rebase). `ORIG_HEAD` = la position de ta branche **juste avant** la dernière opération risquée.

Exercice 1 — Merge dans le mauvais sens

◎ **Scénario :** par erreur, on a fait `git merge master` **depuis develop** (master fusionné dans develop), alors que l'énoncé voulait develop **dans** master. Annuler puis refaire dans le bon sens.

▶ Séquence :

```
git log --graph --oneline --all    # 1. constater le merge erroné (on est sur develop)
git reset --hard ORIG_HEAD         # 2. annuler le merge : develop retrouve son état d'avant
git switch master                  # 3. se placer sur la branche RECEVEUSE
git merge --no-ff develop          # 4. merger develop DANS master (bon sens)
git log --graph --oneline          # 5. vérifier le commit de fusion
```

★ À retenir :

- `--no-ff` (no fast-forward) : force la création d'un **commit de merge** même si Git aurait pu se contenter d'avancer le pointeur. Ça garde la trace visuelle « ces commits venaient d'une branche ».
- Juste après un merge, `ORIG_HEAD` = l'état d'avant le merge → annulation en une commande.

Exercice 2 — Rebase dans le mauvais sens

◎ **Scénario :** sur master, on a fait `git rebase develop` par erreur (master replanté sur develop) — l'inverse du but. Annuler, puis rebaser correctement.

▶ Séquence :

```
git log --graph --oneline --all    # 1. constater : master contient les commits de develop
git reset --hard ORIG_HEAD         # 2. annuler le rebase erroné (on est sur master)
git switch develop                 # 3. se placer sur develop
git rebase master                  # 4. BON sens : develop se replante au bout de master
git log --graph --oneline --all    # 5. vérifier : historique linéaire, develop au-dessus de master
```

★ À retenir :

- Un rebase erroné s'annule **exactement comme** un merge erroné : `reset --hard ORIG_HEAD`.
- Après le bon rebase : develop = commits de master + copies de ses propres commits (nouveaux sha, cf. loi n°1 du récap).

Exercice 3 — Commits faits sur master au lieu de develop

◎ **Scénario :** 2 commits ont été faits sur `master` par erreur, ils auraient dû aller sur `develop` (qui pointe 2 commits en arrière). Déplacer les commits sur la bonne branche SANS les recréer.

▶ Séquence :

```
git log --graph --oneline --all    # 1. constater : les 2 commits sont au bout de master
git branch -f develop HEAD         # 2. avancer develop jusqu'ici (elle englobe les 2 commits)
git reset --hard HEAD~2            # 3. reculer master de 2 commits (les commit s'en vont de master)
git log --graph --oneline --all    # 4. vérifier : les commits n'existent QUE sur develop
```

★ À retenir :

- **`git branch -f develop HEAD`** déplace une branche **sans la checkout** : develop saute jusqu'au commit courant.
- **`HEAD~2`** = « 2 commits avant HEAD » (syntaxe `~n`). C'est le reset dans l'autre sens : ici on recule master plutôt que d'avancer develop.
- Astuce : comme les 2 commits sont les mêmes objets, aucune copie, aucun nouveau sha — on n'a fait que **déplacer deux pointeurs**. C'est ce qui rend la manœuvre propre.
- Ordre crucial : d'abord **`branch -f`** (sauver les commits), ensuite le **`reset`** (les lâcher de master). Inversé = commits orphelins (récupérables via reflog, mais stress évitable :)).

◆ Mémo des 3 modes de reset

Option	Pointeur	Staging	Fichiers
`--soft`	bouge	gardé	gardés
`--mixed` (défaut)	bouge	vidé	gardés
`--hard`	bouge	vidé	**écrasés** ⚠

Règle d'or : `--hard` seulement quand le working tree est propre (ou modifs acceptées comme perdues).

TP 3 — Réécrire l'historique (git rebase -i)

> **Principe du rebase interactif :** `git rebase -i` ouvre un **fichier « todo »** listant les commits du plus ancien (en haut) au plus récent (en bas). Tu édites l'action devant chaque ligne, Git rejoue la liste. ⚠ Dans le todo, l'ordre est **inversé** par rapport à `git log` (le plus ancien en HAUT).

Les actions disponibles (le vocabulaire à connaître)

Action	Effet
`pick`	garder le commit tel quel
`reword`	garder le commit, mais **changer son message**
`squash`	**fusionner** ce commit dans celui de la ligne DU DESSUS
`drop`	**supprimer** le commit ET ses changements
`edit`	rejouer jusqu'ici, faire une **pause** pour retoucher, puis `--continue`
(réordonner)	déplacer les lignes = rejoue les commits dans l'ordre de la liste

---

Exercice 1 — Modifier et fusionner (les Clegane)

◎ **Scénario :** historique GoT : « ajoute le Limier » → « ajoute la Montagne » → revert du Limier. Objectif : fusionner les deux ajouts en un seul commit **« ajout des frères Clegane »**, et renommer le revert en **« retirer le limier »**.

▶ Séquence :

```
git log --oneline             # 1. voir l'historique de départ
git rebase -i --root          # 2. rebase depuis la RACINE (inclut le 1er commit)
#    dans le todo :
#      pick   xxx ajoute le limier
#      squash yyy ajoute la montagne   → fond dans le commit du dessus
#      reword zzz revert ...           → renommer en "retirer le limier"
git log --oneline             # 3. vérifier : "ajout des frères Clegane" + "retirer le limier"
```

★ À retenir :

- `--root` : remonte jusqu'au tout premier commit (sinon le commit initial est hors de portée).
- `squash` fusionne **vers le haut de la liste** (dans le commit précédent). Git ouvre ensuite un éditeur pour rédiger le message fusionné.
- `reword` : Git ré-ouvre un éditeur sur le message, tu modifies, tu sauvegardes → nouveau sha évidemment.

Exercice 2 — Supprimer et inverser (Cersei, Joffrey, Tywin)

◎ **Scénario :** commits Tywin → Cersei → Joffrey. Objectif : **supprimer Cersei** et remettre **Joffrey avant Tywin**.

▶ Séquence :

```
git log --oneline              # 1. état initial : Tywin, Cersei, Joffrey
git rebase -i --root           # 2. todo : déplacer la ligne Joffrey AU-DESSUS de Tywin
#                              #    et passer Cersei de pick → drop
git log --oneline              # 3. résultat : initialisation → Joffrey → Tywin (sans Cersei)
```

★ À retenir :

- **`drop`** enlève le commit ET son contenu (≈ le contraire de revert : aucun commit correcteur n'est créé, l'historique est réécrit).
- Réordonner = couper/coller des lignes du todo. Attention : si deux commits touchent les mêmes lignes, l'inversion peut créer des **conflits** (Git s'arrête → résoudre → `git rebase --continue`).

Exercice 3 — Découper un commit (la branche NORD)

◎ **Scénario :** la branche `NORD` a UN gros commit qui mélange 2 choses. Objectif : le **scinder en deux commits propres** (« mort d'oncle Ben » et « mort du lord commander »), puis intégrer NORD dans master.

▶ Séquence :

```
git switch NORD
git rebase -i master           # 1. todo : passer le gros commit en edit
#    → Git rejoue puis fait une PAUSE juste après ce commit

git reset HEAD^                # 2. annuler LE commit (mixed) : fichiers conservés, commit parti
#    ...recommiter en deux fois, partie par partie :
git add <partie 1> && git commit -m "mort d'oncle Ben"              # 3.
git add <partie 2> && git commit -m "mort du lord commander"        # 4.
git rebase --continue          # 5. finir le rebase

git switch master              # 6.
git merge NORD                 # 7. intégrer la branche découpée proprement
git log --oneline              # 8. vérifier les deux sous-commits
```

★ À retenir :

- **`edit`** + **`git reset HEAD^`** **(mixed)** est LA recette du découpage : on « défait » le commit sans perdre son contenu, puis on re-commit en plusieurs morceaux.
- Ici on veut garder les fichiers → **jamais de `--hard`** (on utiliserait `--soft` si on voulait tout restagé d'un coup, `--mixed` met tout en non-stagé).
- ⚠ Réécrire l'historique = nouveaux sha → **jamais sur des commits déjà poussés/partagés** (ou alors `push --force` assumé et équipe prévenue).

◆ Mémo rebase interactif

```
git rebase -i master       # retravailler les commits de ma branche depuis master
git rebase -i --root       # retravailler TOUT, y compris le commit initial
git rebase --continue      # poursuivre après une pause (conflit résolu ou edit terminé)
git rebase --abort         # abandonner, retour à l'état d'avant le rebase
```

TP 4 — Déboguer avec Git (bisect & blame)

> **L'idée de `git bisect` :** tu donnes un commit « bon » (le bug n'y est pas) et un commit « mauvais » (le bug y est). Git fait une **recherche dichotomique** : il te place au milieu, tu testes, tu dis `good` ou `bad`, et ainsi de suite. En ~$\log_2(n)$ tests, il désigne le **premier commit fautif**. Avec 1000 commits, ~10 tests suffisent.

Exercice 1 — Bisection manuelle + blame

◎ **Scénario :** le site actuel est buggé alors que la version `SITE_V02` était saine. Trouver **le commit exact** qui a introduit le bug, puis identifier **qui** a écrit la ligne fautive.

▶ Séquence :

```
git bisect start               # 1. lancer la bisection
git bisect bad                 # 2. HEAD = mauvais (le bug est présent)
git bisect good SITE_V02       # 3. le tag SITE_V02 = bon (pas de bug)
#    → Git checkout automatiquement un commit AU MILIEU

ls images/                     # 4. TESTER : la médaille d'argent est-elle là ? (ou ouvrir le site)
git bisect bad                 # 5a. bug présent → bad
#    ou
git bisect good                # 5b. bug absent → good
#    → Git recoupe l'intervalle en deux ; RÉPÉTER 4–5 jusqu'à :
#    "4116a99 ... is the first bad commit: ajoute la médaille d'argent" ✔

git bisect reset               # 6. ⚠ OBLIGATOIRE : quitter la bisection, retour sur master
git blame hello.html           # 7. qui a écrit chaque ligne ? → dernière modif : Alice Martin
```

★ À retenir :

- Pendant la bisection tu es en **HEAD détachée** (normal) — d'où l'importance du **`git bisect reset`** final pour revenir proprement sur ta branche.
- `bisect` trouve **QUEL** commit a introduit le bug ; `blame` dit **QUI** a touché chaque ligne (et dans quel commit). Les deux se complètent.
- Réponds toujours `good` / `bad` par rapport à **la présence du bug**, pas à la qualité du code :)

Exercice 2 — Bisection automatisée (`git bisect run`)

◎ **Scénario :** même chasse au bug, mais le test est faisable par un **script** → laisser Git enchaîner les tests tout seul.

▶ Séquence :

```
# 1. Écrire le script de test — protocole bisect :
#    exit 0 = GOOD (bug absent) ; exit 1-127 (sauf 125) = BAD (bug présent) ; exit 125 = skip
cat > test-medal-silver.sh <<'EOF'
#!/bin/sh
```

```
# Le bug = présence de la médaille d'argent
# le script réussit (0) quand le fichier est ABSENT → bon
test ! -f images/medal_silver.png
EOF
chmod +x test-medal-silver.sh  # 2. le rendre exécutable

git bisect start               # 3.
git bisect bad                 # 4. HEAD buggé
git bisect good SITE_V02       # 5. V02 saine
git bisect run ./test-medal-silver.sh   # 6. Git joue seul : checkout → test → good/bad → ...
#    → trouve le même premier commit fautif, sans intervention ✔

git bisect reset               # 7. nettoyer, retour sur master
```

★ À retenir :

- **Le protocole du script** est à connaître par cœur :

Code exit du script	Interprétation bisect
`0`	commit GOOD (bug absent)
`1` à `124`, `126`, `127`	commit BAD (bug présent)
`125`	SKIP (impossible à tester, ex: ne compile pas)

- Pense à l'**inversement logique** de l'exo : le fichier absent = bon → `test ! -f` renvoie 0 quand le fichier n'existe pas. ✔
- Mets le script **hors du dépôt** ou en dehors des fichiers suivis si possible : la bisection change le contenu du working tree à chaque étape (un script tracké pourrait changer/disparaître en cours de route).
- `blame` bonus : `git blame -L 10,20 hello.html` cible des lignes précises ; `git log -S"mot"` cherche le commit qui a ajouté un texte (« pickaxe »).

◆ Mémo bisect

```
git bisect start          # démarre
git bisect bad [<ref>]    # marque un commit mauvais (défaut : HEAD)
git bisect good <ref>     # marque un commit bon
git bisect run <script>   # automatise
git bisect reset          # termine et ramène sur ta branche — à ne JAMAIS oublier
```

TP 5 — Les hooks Git

> **Un hook = un script que Git exécute automatiquement** à un événement précis (avant un commit, à la réception d'un push...). Emplacement : `.git/hooks/`. Deux règles vitales : le fichier doit s'appeler **exactement** comme l'événement (`pre-commit`, `commit-msg`...) et être **exécutable** (`chmod +x`). Si le script sort avec un **exit ≠ 0**, l'opération est **bloquée**.

Exercice 1 — Hooks locaux (côté client)

◎ **Scénario :** imposer deux règles sur les commits locaux : (1) le dossier `images/` doit exister avant de committer, (2) le message doit commencer par `id: message`.

▶ Séquence :

```
# 1. Hook pre-commit : vérifié AVANT la création du commit
cat > .git/hooks/pre-commit <<'EOF'
#!/bin/sh
if [ ! -d ./images ]; then
  echo "ERREUR : le dossier ./images n'existe pas"
  exit 1                      # exit ≠ 0 → commit REFUSÉ
fi
EOF

# 2. Hook commit-msg : vérifié sur le MESSAGE (Git passe le fichier du message en argument $1)
cat > .git/hooks/commit-msg <<'EOF'
#!/bin/sh
if ! grep -qE '^[0-9]+: .+' "$1"; then
  echo "ERREUR : le message doit être de la forme 'id: message'"
  exit 1                      # exit ≠ 0 → commit REFUSÉ
fi
EOF

# 3. ⚠ Sans exécutabilité, Git IGNORE les hooks silencieusement
chmod +x .git/hooks/pre-commit .git/hooks/commit-msg

# 4. Tests attendus :
git commit -m "test"                  # ✘ refusé (pas de dossier images / pas d'id)
mkdir images && git add images ...    # préparer un commit conforme
git commit -m "ajoute les images"     # ✘ refusé (pas d'id numérique)
git commit -m "123: ajoute les images"  # ✔ accepté
```

★ À retenir :

- **`grep -qE 'regex' "$1"`** = test silencieux d'une expression régulière sur le contenu du message. `^[0-9]+:` = commence par un nombre, deux-points, espace.
- Les hooks locaux vivent dans `.git/` → **non versionnés, non poussés** : chaque développeur doit les installer chez lui (d'où l'intérêt des hooks serveur).

Exercice 2 — Hook serveur : le hook `update`

◎ **Scénario :** sur le dépôt distant (un dépôt **nu** `server.git`), interdire les tags **légers**, autoriser les tags **annotés**, et bloquer la **suppression de tags** sauf autorisation explicite.

▶ Séquence :

```
# 1. Créer le "serveur" : dépôt NU (bare) = sans working tree, ne sert qu'à recevoir des push
git init --bare server.git

# 2. Installer le hook update (adapté de hooks/update.sample)
#    il reçoit 3 arguments : $1=nom de la ref, $2=ancien sha, $3=nouveau sha
#    et lit la config : hooks.allowunannotated, hooks.allowdeletetag
cp server.git/hooks/update.sample  # base de travail
#    → éditer adapter → enregistrer comme server.git/hooks/update
chmod +x server.git/hooks/update
git -C server.git config hooks.allowunannotated false   # tags légers interdits
git -C server.git config hooks.allowdeletetag false     # suppression de tags interdite

# 3. Seed : un client pousse un premier commit
git clone server.git client && cd client
git add . && git commit -m "premier commit" && git push origin master

# 4. Tests depuis le client :
git tag v1                          # tag LÉGER
git push origin v1                  # ✘ REFUSÉ par le hook update
git tag -d v1                       # nettoyer le tag léger

git tag -a v1 -m "Version 1"        # tag ANNOTÉ
git push origin v1                  # ✔ ACCEPTÉ

git push origin :refs/tags/v1       # tentative de SUPPRESSION distante
                                    # ✘ REFUSÉE (allowdeletetag=false)
git -C ../server.git config hooks.allowdeletetag true   # autorisation explicite
git push origin :refs/tags/v1       # ✔ suppression ACCEPTÉE

git ls-remote --tags ../server.git # 5. vérifier l'état final des tags sur le serveur
```

★ À retenir :

1. **Tag léger vs annoté** : léger = simple alias vers un sha ; annoté = vrai objet Git avec auteur, date, message → **seul acceptable pour une release**, d'où la politique du serveur.
2. **Hook `update`** = garde-frontière du serveur : il s'exécute **par référence poussée** et peut refuser n'importe quel push → protège toute l'équipe, pas juste ta machine.
3. `git push origin :refs/tags/v1` (avec les `:`) = syntaxe de suppression d'une ref distante (équivalent : `git push origin --delete v1`).
4. Hiérarchie des lieux de contrôle : **client** (pre-commit, commit-msg : rappels locaux, contournables) vs **serveur** (update : loi non contournable).
5. `-C <dossier>` permet de lancer git **dans** un autre dépôt sans `cd`.

◆ Mémo hooks

Hook	Où	Quand	Usage typique
`pre-commit`	client	avant la création du commit	lint, fichiers requis
`commit-msg`	client	après saisie du message	format du message (ticket n°…)
`pre-push`	client	avant un push	lancer les tests
`update`	serveur	à chaque ref poussée	politique de tags/branches
`post-receive`	serveur	après réception	déploiement, notification

TP 6 — Sous-modules / multi-dépôts

> **Un sous-module = un dépôt Git indépendant embarqué dans un dépôt parent.** Le parent ne stocke **que deux choses** : l'URL du sous-module (dans `.gitmodules`) et **le sha exact** du commit du sous-module (un pointeur). Le parent ne contient PAS les fichiers du sous-module.

Exercice — MON_SITE + TC_CSS + TC_IMAGES

◎ **Scénario :** le site `MON_SITE` utilise les styles du dépôt `TC_CSS` (déjà en sous-module) et doit embarquer `TC_IMAGES` en 2e sous-module. Il faut afficher la médaille d'argent, ajouter la médaille d'or **dans le dépôt images**, mettre à jour le parent, et vérifier qu'un clone frais reçoit tout.

▶ Séquence :

```
# 1. Cloner le projet principal ET ses sous-modules existants
git -c protocol.file.allow=always clone --recurse-submodules /chemin/MON_SITE
#    (sans --recurse-submodules : git submodule update --init ensuite)

cd MON_SITE
ls css/                          # 2. vérifier : le sous-module CSS est rempli ✔

# 3. Ajouter TC_IMAGES comme sous-module dans ./images
git -c protocol.file.allow=always submodule add ../TC_IMAGES images
git add .gitmodules images       # 4. .gitmodules + pointeur du sous-module
git commit -m "ajoute le sous-module images"

# 5. Page avec médaille d'ARGENT (les fichiers de TC_IMAGES sont dispo)
#    → éditer hello.html, commit d'usage éventuel

# 6. Ajouter la médaille d'OR : TRAVAILLER DANS le dépôt du sous-module
cd images
git add medal_gold.png
git commit -m "ajoute la médaille d'or"
git push                         # ⚠ pousser le SOUS-MODULE d'abord !
cd ..

# 7. Le parent enregistre le NOUVEAU pointeur du sous-module
git add images                   # stocke le nouveau sha de TC_IMAGES
git commit -m "met à jour le sous-module images (médaille d'or)"
#    → éditer hello.html pour afficher l'or, commit
git push                         # 8. pousser le parent APRÈS le sous-module

# 9. VÉRIFICATION : un collègue clone tout frais
git -c protocol.file.allow=always clone --recurse-submodules /chemin/MON_SITE clone2
ls clone2/css clone2/images      # ✔ les 2 sous-modules remplis, medal_gold.png présent
```

★ À retenir :

1. **Ordre des push : sous-module AVANT parent.** Si le parent pointe vers un sha que le serveur du sous-module ne connaît pas → les collègues clonent un dossier d'images vide/cassé.
2. `git submodule add <url> <chemin>` crée `.gitmodules` + un **gitlink** (le pointeur sha) : les deux se commitent dans le parent.

3. Après un clone classique, les dossiers des sous-modules sont **VIDES** → `git submodule update --init` (ou cloner d'emblée avec `--recurse-submodules`).
4. **`-c protocol.file.allow=always`** : option ponctuelle nécessaire en TP car les remotes sont des dépôts locaux (`file://`), bloqués par défaut pour les sous-modules depuis Git 2.38 (sécurité). Inutile avec de vraies URLs https/ssh.
5. Dans le parent, `git add images` semble étrange (un dossier « vide » ?) : non — on met à jour **le sha pointé**, c'est le cœur du mécanisme.

◆ Mémo sous-modules

```
git submodule add <url> <chemin>        # ajouter un sous-module
git submodule update --init             # remplir les sous-modules après un clone normal
git clone --recurse-submodules <url>    # clone « tout compris »
git submodule status                    # sha pointés vs sha réels
cd <sous-module> && git pull            # mettre à jour le CONTENU du sous-module
git diff --submodule                    # voir le changement de pointeur
```